Chapter 5

Working with a Relational DBMS

IN THIS CHAPTER

- **Understanding RDBMS characteristics**
- **Working with RDBMS data**
- **Developing datasets from RDBMS data**
- **Using more than one RDBMS product**

Relational databases accomplish both the manipulation and data retrieval objectives with relative ease. However, because data storage needs come in all shapes and sizes for a wide range of computing platforms, many different relational database products exist. In fact, for the data scientist, the proliferation of different Relational Database Management Systems (RDBMSs) using various data layouts is one of the main problems you encounter with creating a comprehensive dataset for analysis. The first part of this chapter focuses on common characteristics of RDBMS so that you have a better idea of what commonality you can expect when working with them.

The second part of the chapter discusses the mechanics of working with an RDBMS. However, in contrast to how code is done in many other chapters in this book, writing functional code is nearly impossible in this case because of variations between RDBMS and the fact that

we, as the authors, can't be sure what RDBMS you have at your disposal. Trying to create a functional example can't be done in these circumstances, but you do receive enough information to work with your product to create your own RDBMS-specific applications.

Interacting with an RDBMS is complex, but it's only the first step in a longer process. The third part of the chapter discusses what you need to know to move the data from the RDBMS into an environment in which you can perform analysis on it. The fact that RDBMSs vary in characteristics is exacerbated by the significant differences in relational design that have plagued Database Administrators (DBAs) for years. You can't even be sure at the outset about table design differences between data sources, which is a considerable problem given that you must combine tables to obtain a complete picture of the data.

Finally, RDBMS vendors do make a huge effort to differentiate their products. Yes, standards exist, but vendors also have a strong incentive to add functionality that makes their product better than the competition in some way, which is a problem for the data scientist because now you have to deal with those differences when making various products work together. The final part of the chapter discusses a few of these issues.

Considering RDBMS Issues

Chapter 4 of this minibook discusses various sorts of flat-file databases. In most cases, these databases contain just one table, and you rely on just one file for the entire database. An exception might be a specialty database of the sort used by point-of-sale (POS) systems, but generally, you have only one thing to worry about.

REMEMBER An RDBMS can handle extremely complex data because it relies on Edgar (Ted) Frank Codd's (`https://history-`

computer.com/ModernComputer/Software/Codd.html) relational rules, as described at https://www.w3resource.com/sql/sql-basic/codd-12-rule-relation.php . An RDBMS, by definition, relates data in multiple tables together so that you can store a great deal of information in an incredibly small space while still making that data accessible with less effort than you might normally require.

JUST HOW COMPLEX IS COMPLEX?

To give you some idea of just how complex an RDBMS can become, consider Microsoft's AdventureWorks database (http://merc.tv/img/fig/AdventureWorks2008.gif), which is designed to mimic the real world. The diagram shows all the tables that this database contains. This database requires a great deal of commentary to even start to understand (see one such commentary at http://www.wilsonmar.com/sql_adventureworks.htm), and there are databases that are considerably more complex than this one. Unlike most forms of data storage discussed in this book, the RDBMS gives you the best idea of what you might be facing when developing a truly complex real-world analysis.

The following sections offer an extremely brief overview of RDBMS characteristics as they apply to data science. If you were to perform an analysis based on information from a complex RDBMS, you would probably require the services of a skilled DBA to help you. Yes, the data can become that complicated.

Defining the use of tables

A single table contains just one sort of information, much as a flat-file database does. However, unlike a flat-file database, the single table may be part of a much larger picture. For example, consider an employee entry in a database. The employee information table may contain just *one-off information* (those entries that consume just one

record), such as name and employee number. To locate the employee's contact information, you may need to look at another table that records just contact information.

An employee could have multiple contacts — such as multiple addresses, multiple phone numbers, and so on — so a one-to-many relationship exists between the employee information table and the contacts table. The relation between the single employee entry and the multiple contact information records is what an RDBMS is all about. You enter each piece of information only once. To obtain a complete record for analysis purposes, you need both tables (and often a great many more).

REMEMBER When you're a DBA, you need to understand the relationships between all the tables. Even a manager might need to understand the fact that multiple tables constitute a single employee record. A data scientist may not be interested in all the tables and may not even want all of them. In fact, if your sole purpose is to create an analysis of where employees live (to help city planners develop travel patterns, for example, as part of an infrastructure upgrade), you don't even need the entire contacts table. Perhaps all you really need is the locality, city, and state for each of the employees. You don't want the information to be personally identifiable, so you don't even want the employee number that would normally be part of that table (to create the relationship between the employee and the contact).

Unfortunately, as a data scientist, the contacts table may not actually contain everything you need. What you need is a locality, not a precise address. The contacts table contains a precise address that you would then need to cross-reference to a locality, say the Port View area of town. The resolution of the information depends on the purpose to which you put it. A fine-grained resolution of precise address may ac-

tually make the analysis harder than it needs to be and provide less useful information. Now you need:

- Part of the contacts table
- Nothing from the employee information table (except to create the initial data request)
- An external database that provides localities based on the precise address

TIP Often, when you start looking at data for analysis, what you find is that it's presenting a view of something that satisfies someone else's needs, not yours. Before you make an initial data request and start cleaning the data, you need to work through how you plan to use the data in your analysis, which may mean spending considerable time looking at a diagram of the database schema. The database schema will show things like:

- How one table is linked to another
- Which fields link to other fields to create the relationship
- The type of relationship (such as one-to-many, one-to-one, or many-to-many)
- Fields used for indexing
- Fields used as a key for searching

Understanding keys and indexes

For data scientists, keys and indexes take on a different meaning than they might for other RDBMS users. However, the use of these two elements is the same. Multiple kinds of keys exist, as described here:

- **Primary key:** One or more table columns that uniquely identify each row in the table. Unlike other kinds of databases, an RDBMS contains code that ensures that each primary key is unique. For data scientists, the use of a primary key means that you can count on the uniqueness of each record. In addition, primary keys can't contain missing elements, so you can be sure that the information you receive from a primary key is always complete, but not necessarily correct.

- **Foreign key:** An entry in a secondary table that points to a primary key in another table. For example, a contacts table will contain a foreign key containing the employee ID of an employee in an employee information table. The key is unique in the employee information table, but you may see multiple copies of the key in the contacts table — each of which points at a particular employee.
- **Other keys:** Depending on the resource you use, you may see terms such as super key, composite key, alternate key, and so on. For the most part, as a data scientist, you really don't care about these other keys unless they somehow affect how you retrieve information from the RDBMS.

As you can see from the list, a *key* is part of a mechanism to uniquely identify records. Because the keys are unique, you can generally use the indexes created from them to search for a single result in the table when working with a primary key, to search for or a group of related entries when working with a foreign key.

An *index* is part of a mechanism to make searching for data efficient, regardless of whether the data is unique. For example, you can create an index that views telephone numbers based on area code. The area code isn't unique, but you can search on the area code to locate all the numbers that use it. The goal is to find information groups rather than unique information.

For purists, a key is part of the database's logical structure, not an entity. When you create a key, the RDBMS also creates an index for you based on that key. You can also create indexes that aren't based on a key. So, you always search using an index, even you appear to be using a key to do it. This can become a confusing issue for some, and for a data scientist, it doesn't really matter. All that matters is locating all the records in the RDBMS that meet certain criteria.

Using local versus online databases

In general, an RDBMS works the same whether you access it locally or remotely. You still obtain data based on a query in the form that the RDBMS is designed to provide and within the constraints that you specify. However, some mechanics of working with online databases differ from those experienced when working locally:

- The method of access will likely differ, and you may need to configure elements like firewalls to work with the RDBMS.
- Access times will increase when working remotely, and response times may be longer still.
- An RDBMS may limit the forms of response data to those that work well with a remote connection, meaning that you may need to perform additional data transformations.
- A remote connection may experience limits in data access for security reasons.

You may find additional differences between local and online access, but these are the most common. The point is that, when working with large amounts of sensitive data, a local connection may prove more useful. Even if you make the request, place the data on a hard drive, and move it to the remote location, you may still find that you save time and effort. Oddly enough, this is a common practice even with large online services such as AWS (see `https://aws.amazon.com/blogs/aws/send-us-that-data/` for details).

Working in read-only mode

An RDBMS devotes considerable resources to keeping data safe, as described by Codd's 12 rules. However, you can reduce the overhead of interacting with data simply by telling the RDBMS that you have no desire to modify anything. In fact, as a data scientist, you really don't want to modify anything most of the time, so opening the database in full read/write mode makes no sense whatsoever. Consequently, by opening your database in read-only mode, you can gain a speed advantage in performing various tasks.

TIP A second level of read-only is to create a database snapshot. A snapshot takes a picture of the database at a specific time so that you have static data. Chapter 1 of this minibook discusses the use of static and dynamic data sources. However, in this case, the choice has significant consequences. When you choose to access an RDBMS database in read-only mode, you still have the database engine performing updates in the background, costing you time. If you use a snapshot, no updates take place and you can access data much faster.

Of course, read-only mode and the use of snapshots increase data access speed at the cost of interactivity. You must first consider how you need to interact with the database (normally you don't want to change anything; you just want to acquire data for analysis). Then you must consider whether the data changes quickly enough to warrant using a dynamic setup.

Accessing the RDBMS Data

Understanding how an RDBMS works from the data science perspective enables you to access the database in an efficient manner. However, depending on the data you need, you may need to rely on several techniques to access it. An RDBMS doesn't simply spit out a file with the information you need; you need to understand how to request the data in a manner that reduces the number of requests while increasing the number of records you receive that actually match your requirements. The following sections discuss a few of the most common data access methods.

KEEPING REQUESTS UNDER CONTROL

The temptation exists to request every possible record with every possible field to reduce the number of requests that an application makes from an RDBMS. However, this trap increases resource usage, reduces

response speed, and makes manicuring the data later painful. In a perfect world, you would tailor a request in a manner that retrieves just the records you need and only the fields you actually require for analysis. The world isn't perfect, though, so you normally end up with extra data. However, you can reduce the waste and make your applications run considerably faster by reducing the amount of data you request.

In addition, you must also choose the correct request technique. The more work you can perform on the server, the less you have to transport across the network (no matter what kind of network you use). However, you need to keep in mind that working on the server keeps you from seeing the entire dataset, so this approach involves a trade-off. You may actually end up missing data that would make your analysis better by putting too many restrictions on the request. In the end, making requests is more art than science, and you gain a good understanding of what to do only through practice.

Using the SQL language

The one common denominator among many relational databases is that they all rely on a form of the same language to perform data manipulation, which does make the data scientist's job easier. The Structured Query Language (SQL) lets you perform all sorts of management tasks in a relational database, retrieve data as needed, and even shape it in a particular way so that the need to perform additional shaping is unnecessary.

Creating a connection to a database can be a complex undertaking. For one thing, you need to know how to connect to that particular database. However, you can divide the process into smaller pieces. The first step is to gain access to the database engine using a product like SQLAlchemy (`https://www.sqlalchemy.org/`). You use two lines of code similar to the following code. (Note, however, that the code presented here is not meant to execute and perform a task; the examples

at `https://docs.sqlalchemy.org/en/13/core/engines.html` will help you in this regard.)

```
from sqlalchemy import create_engine
```

```
engine = create_engine('sqlite:///:memory:')
```

After you have access to an engine, you can use the engine to perform tasks specific to that DBMS. The output of a read method is always a `DataFrame` object that contains the requested data. To write data, you must create a `DataFrame` object or use an existing `DataFrame` object. You normally use these methods to perform most tasks:

- `read_sql_table()`: Reads data from a SQL table to a `DataFrame` object
- `read_sql_query()`: Reads data from a database using a SQL query to a `DataFrame` object
- `read_sql()`: Reads data from either a SQL table or query to a `DataFrame` object
- `DataFrame.to_sql()`: Writes the content of a `DataFrame` object to the specified tables in the database

The `sqlalchemy` library provides support for a broad range of SQL databases. The following list contains just a few of them:

- SQLite
- MySQL
- PostgreSQL
- SQL Server
- Other relational databases, such as those you can connect to using Open Database Connectivity (ODBC)

You can discover more about working with databases using SQLAlchemy at `https://docs.sqlalchemy.org/en/13/core/tutorial.html`. This tutorial helps you through some of the unique elements of different vendor offerings with regard to accessing data. The techniques that you discover in this book for using the toy databases also work with

RDBMSs. However, these principles apply only to retrieving and manicuring the data. If you want to start creating database objects, updating tables, and performing other database tasks, you must follow the SQLAlchemy techniques for doing so because they don't quite match what you would do within the database manager. In addition, you also need to know about differences between vendor SQL implementations to make the code work properly. If you need to expand your knowledge of the SQL language, check out *SQL All-in-One For Dummies* by Allen G Taylor (Wiley).

OPENING A FIREWALL PORT

One of the issues you may have to consider when working with an RDBMS is that the request port differs from the standard port 80 or 443 (for SSL) used for many requests. These special ports often vary by vendor and product. For example, the default SQL Server port is 1433, which means reconfiguring your firewall to make an exception (`https://docs.microsoft.com/en-us/sql/sql-server/install/configure-the-windows-firewall-to-allow-sql-server-access?view=sql-server-2017`). Even when using a local network setup, you must make the change in the firewall configuration, which can become a sticking point in some cases when opening a port is an issue for support staff.

Relying on scripts

A *script* (sometimes called a stored procedure) is like a mini-application that exists within the RDBMS environment. For many RDBMSs, a script is merely a series of SQL statements bound together by a little glue code. The advantage of working with scripts is that you can obtain relatively precise results because the script will perform a substantial amount of data manipulation for you. In addition, with the use of the right arguments, you can make the script quite flexible so that it can answer a wide variety of needs.

Using a script means that you can perform at least some of the processing needed to obtain useful data on the server rather than on the client. The problem is figuring out how to create a script that provides a generic enough response, yet delivers the data you need. The tutorial at `https://www.w3schools.com/sql/` gives the basics of creating SQL statements, and the tutorial at `https://docs.microsoft.com/en-us/sql/ssms/tutorials/scripting-ssms?view=sql-server-2017` helps you understand the tool used to create and manage scripts, SQL Server Management Studio (SSMS), for Microsoft's product.

REMEMBER Unfortunately, scripting is an area in which the RDBMS characteristics tend to vary by vendor. The script you create for SQL Server is unlikely to run on MySQL. Consequently, you need to know each RDBMS or work with a DBA for that RDBMS to create the custom script you require. The disadvantage of using scripts is complexity. This is a platform-specific solution that requires substantial time to hone in many cases.

Relying on views

You can frequently use views to obtain a particular piece of data you need on a regular basis. A *view* is the stored result of a SQL query. As a result, it represents a static dataset, but you can rerun the view to update the information. A view differs from a *snapshot* in that a snapshot isn't updateable and a snapshot is a static image of the database as a whole. (Depending on which resource you use, you may find conflicting definitions of *view* and *snapshot,* but these are the definitions used for this book.) When you download a view, you see just the piece of the database reflected by the SQL query, while a snapshot would provide raw information that you could query locally in ways that you hadn't originally intended with the view.

Using views is very efficient because you transfer only the data you need. Unlike scripts, views tend to be generic across platforms because most platforms use a standard set of SQL commands (with some small modifications as the commands become more complex). You require less intimate knowledge of the platform to use a view, and a DBA who is reluctant to let you create a script (because they are more powerful and flexible) may let you create a view, which can't modify the database content.

REMEMBER The negative part of using views is that they're single use: The same query means the same result unless the underlying data changes. In addition, views are significantly less powerful than scripts, so using one means that you spend more time manicuring the data locally after you receive it. What you gain with views is ease of use, but that gain comes with a loss of flexibility and efficiency.

Relying on functions

A *function* in an RDBMS works like a function in functional programming (see Chapter 2 of this minibook) because SQL is a declarative programming environment. Functions differ from scripts in a number of important ways. Functions

- Can't modify state (including database records)
- Can be used to calculate values
- Must return a value (scripts can simply perform tasks without returning a value)
- Can appear as part of SQL statements when they return a scalar value
- Can't call scripts (but scripts can call functions)

Other differences likely exist between scripts and functions depending on the RDBMS vendor, but these are the important differences from a data science perspective. The two most important of these differences are that a function won't modify state (which is good because you

don't usually want to modify state, even accidentally) and you can use a function to calculate values.

TIP The ability to calculate values and then use that calculation as part of a request is extremely important. For example, you could perform part of your statistical analysis on the server using a function and then use the result to make another request without ever passing information back to the client. This means that you gain a significant advantage in analysis efficiency.

Creating a Dataset

When working with a product such as SQLAlchemy (see the "Using the SQL language" section of the chapter), you use standard Python structures, along with those provided by libraries such as NumPy and pandas. For example, when you make a query, you retrieve a `DataFrame` that looks like any other `DataFrame` you might use with the toy databases used in this book. Consequently, much of what you do with SQLAlchemy output is the same as what you do with any `DataFrame` you have already used. However, the following sections point out some complexities that you need to consider when interacting with an RDBMS that you may not have to deal with otherwise.

Combining data from multiple tables

No matter how hard you try to complete queries on the server, you often have to combine data from queries on the client to create a complete dataset. The important thing is to try to get the database server to do as much of the work for you as possible by creating SQL queries that retrieve precisely what you need. Otherwise, you can spend considerable time trying to determine how to perform the task locally us-

ing two `DataFrame` objects in a manner that you might not otherwise use.

WARNING If you do end up having to combine tables locally, you must do so with a database style merge or join, as described at `https://pandas.pydata.org/pandas-docs/stable/user_guide/merging.html#database-style-dataframe-or-named-series-joining-merging`. If you simply concatenate the two tables, you won't end up with the correct result and your analysis will be tainted.

Unfortunately, you can only join two `DataFrame` objects at a time. If you were to perform the same task on the database server, you could perform one single large join across multiple tables. So, performing the task locally means performing multiple small tasks rather than one big task. To guarantee success, you need to work through a small subset of the data to ensure that you see the correct result before working on the huge datasets normally contained within an RDBMS.

Ensuring data completeness

An RDBMS is a specialized engine that contains all sorts of safeguards not found with your local setup. As you go through the pandas documentation for performing database style merges and joins, note the topics on looking for duplicate keys. When you work with an RDBMS, the RDBMS would automatically detect the duplicate keys and ask how to handle them. This lack of data automation means that you must now perform additional work to manicure your data properly.

Along with duplicate keys, you must consider that you could have to deal with missing keys and missing data. This wouldn't mean that the software is inept, but rather that it lacks the functionality that an RDBMS can provide. In working through a data importation proce-

dure, you must validate attributes such as the data types of the various fields and determine whether the fields are consistent. You perform this kind of work manually because automatic validation is likely to produce inexact results.

TIP At this point, you may wonder why you'd even attempt to import data into one or more `DataFrame` objects and then perform the work locally. The answer goes back to some of the original assumptions at the start of the chapter. The data in the RDBMS may simply not reflect what you need to perform an analysis. For example, if you have precise addresses in the RDBMS, but what you need is localities, you may not have the option of performing all the work in the RDBMS because it simply doesn't have the data you need. Now you find yourself working with multiple data sources, some of which may not even appear in an RDBMS — perhaps the locality information is found in a flat-file database or is the result of an API query.

Slicing and dicing the data as needed

Slicing and dicing the data works much as it does for any Python object (see the "Selecting and Shaping Data" section of Book 2, Chapter 3), except that you must now consider how the data is put together before you do anything. The `DataFrame` you create as the result of various database requests and manipulations may contain complexities that would make normal slicing and dicing strategies inappropriate. For example, cutting off the keys can save space but might also result in an unusable `DataFrame`.

Mixing RDBMS Products

The DBA at your organization is likely using a single product at a complex and detailed level far beyond what you need. The managers are

accessing this data in a manner that helps them perform tasks such as evaluating employees and seeing the latest sales figures. Your RDBMS use will differ from anyone else's at the organization, in many cases, because you need outside data to perform analysis correctly. For example, knowing the sales figures from your organization isn't enough; you may need to know how the compare to other organizations so that you can determine how you're performing against the competition with greater confidence. This difference means that you're quite likely to use more than one RDBMS, each with its own rules.

To determine how best to mix and match the RDBMS products you need, you should review feature comparisons like the one found at `https://medium.com/@mindfiresolutions.usa/a-comparison-between-mysql-vs-ms-sql-server-58b537e474be` for MySQL and SQL Server. Knowing what to expect from each RDBMS will reduce the errors you make when creating queries on each system to retrieve the data you need. In addition, you can start to determine how to deal with RDBMS-specific differences in things like data type handling. One of the more critical issues is how the products handle dates when you perform time-related analysis.

The complexities mount as you begin adding other data sources. If RDBMS isn't complex enough already, you need to consider what happens when you start dealing with flat-file, API, and generated sources. Perhaps some of these sources provide dynamic data when you have a static view or snapshot of the RDBMS data. If so, you need to consider how best to handle that issue. Unfortunately, no best practices exist in this regard. The best approach is probably to experiment to see whether you get what you expected, or if you can explain differences you didn't expect.

Table of contents collapsed